

Android Security Assessment

OWASP UnCrackable App — Level 3

1. introduction

Target: UnCrackable-Level3.apk

Platform: Android

Analysis Type: Static Analysis, Smali Analysis, APK Modification

The assessment focused on:

- Identifying root detection mechanisms.
- Identifying debugger/debuggable detection.
- Identifying native anti-debugging & native password verification
- Recovering the hardcoded secret through static analysis.
- Modifying the application's Smali code to bypass root/debuggable checks.
- Modifying native anti debugging to bypass debuggable checks.
- Rebuilding and signing the modified APK.
- Installing and validating the modified APK in an Android 9 environment.

2. overview

2.1 environment setup

Test Environment: Android 9

Analysis Environment: Kali Linux

The Android application was analyzed inside an isolated Android 9 environment running through Virtual Master. The Kali Linux host communicated with the Android environment through ADB over TCP/IP.

2.2 tools

Tool	Purpose
JADX	Java/decompiled code analysis
Apktool	APK decoding, Smali analysis, rebuilding
apksigner	APK signing

ADB	Device communication and APK installation
Python	Independent verification of cryptographic logic
Virtual Master	Android 9 test environment
Ghidra	Native code analysis

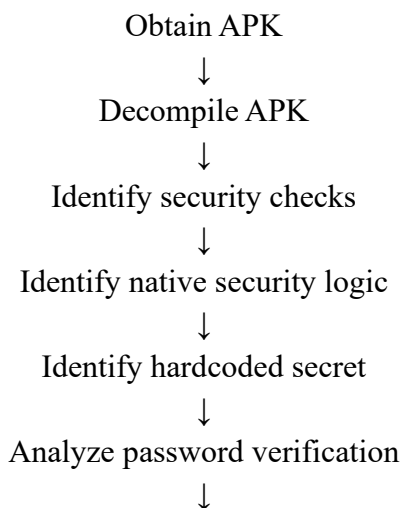
3. findings

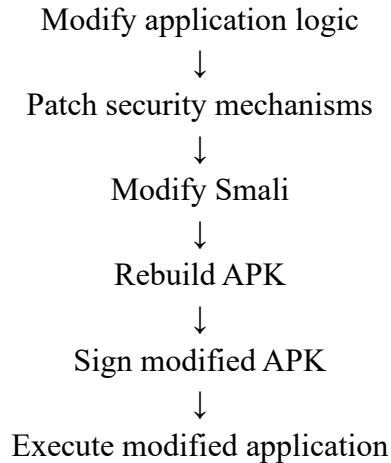
Finding	Severity	Result
Client-side root detection can be bypassed	Medium	Bypassed
Debuggable detection implemented entirely client-side	Low/Medium	Bypassed
Native anti debuggable detection	Low/Medium	Bypassed
Hardcoded secret in native code	High	Recovered
Native password verification	High	Successfully Bypassed

3.1 security impact

The primary security issue is that the application relies on client-side security controls and embedded cryptographic material.

An attacker with access to the APK can:





Therefore, security-sensitive secrets and trust decisions should not rely solely on client-side implementation.

4. Detail Findings

4.1 Initial Static Analysis

The original APK was analyzed using JADX.

The main application components identified were:

- MainActivity
- CodeCheck

Further analysis of the application revealed that important security-sensitive functionality was implemented through native code.

The application loads a native library through:

```
static {  
    System.loadLibrary("foo");  
}
```

This causes Android to load: libfoo.so from the application's native library directory.

The initial static analysis therefore established that the application relies on JNI and native code for password verification and other security-sensitive functionality.

4.2 Native Library Loading

The first indication of native code was identified in MainActivity:

```
static {
```

```
System.loadLibrary("foo");  
}
```

This causes Android to load: libfoo.so from the application's native library directory.

The application therefore depends on JNI to execute important functionality.

4.3 Native Password Verification

The CodeCheck class contains the following native method:

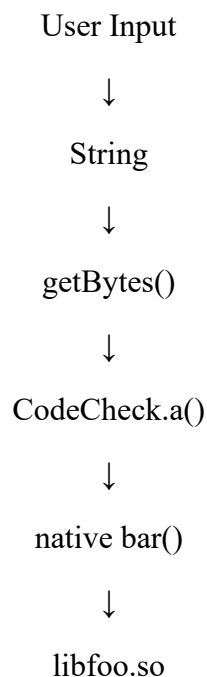
```
private native boolean bar(byte[] bArr);
```

The Java wrapper method was identified as:

```
public boolean a(String str) {  
    return bar(str.getBytes());  
}
```

This reveals that the Java method does not perform the actual password comparison.

Instead, the verification process can be represented as:



The corresponding JNI function was identified as: Java_sg_vantagepoint_uncrackable3_CodeCheck_bar

This function was subsequently analyzed to determine how the native password verification was performed.

4.4 Analysis of bar()

The decompiled native function contained the following important logic:

```
if (DAT_00115054 == 2) {  
    FUN_001010e0(local_68);  
    lVar4 = ... GetByteArrayElements(...);  
    iVar2 = ... GetArrayLength(...);  
    if (iVar2 == 0x18) {  
        ...  
    }  
}
```

The value: 0x18

equals: 24 bytes

Therefore, the application expects the secret to contain exactly 24 bytes.

The comparison loop was especially important:

```
if (*(byte *) (lVar4 + uVar5) != (*(byte *) ((long) &DAT_00115038 + uVar5) ^  
local_68[uVar5]))
```

This can be represented conceptually as: $\text{input}[i] == \text{DAT_00115038}[i] \text{ XOR } \text{local_68}[i]$

for: $i = 0 \dots 23$

The corresponding ARM64 instructions were:

- ldr w10,[x23,x8]
- ldrb w11,[x9,x8]
- ldrb w12,[x21,x8]
- eor w10,w11,w10
- cmp w12,w10
- b.ne LAB_00103474

The important instruction is: eor w10, w11, w10 which performs the XOR operation.

Therefore, the native function is effectively performing: $\text{ExpectedByte} = \text{InternalDataByte} \text{ XOR } \text{GeneratedByte}$; and comparing the resulting value against the user-supplied password.

4.5 Analysis of FUN_001010e0

The function: FUN_001010e0

initially appeared complex because its Ghidra decompilation contained approximately 1,500 lines.

However, further inspection revealed a repeated pattern involving:

- $IVar3 = IVar3 * 0x41c64e6d + 0x3039;$
- $DAT_00115008 = IVar3;$

followed by memory allocation and linked-list manipulation.

Near the end of the function, an important structure was initialized:

- $*param_1 = 0x1549170f1311081d;$
- $param_1[1] = 0x15131d5a1903000d;$
- $param_1[2] = 0x14130817005a0e08;$

This represents 24 bytes of internal data.

The function therefore generates or populates the byte material later used by the XOR-based password verification. The important analytical observation was that the size of the function did not necessarily mean that every instruction was directly relevant to the final password comparison.

4.6 init(byte[]) Analysis

The Java declaration was identified as:

```
private native void init(byte[] bArr);
```

The caller passes:

- `private static final String xorkey = "pizzapizzapizzapizzapizz";`
- `init(xorkey.getBytes());`

Therefore, the native initialization function receives the string: pizzapizzapizzapizzapizz

The corresponding JNI implementation was identified as: `Java_sg_vantagepoint_uncrackable3_MainActivity_init`

The important portion was:

- `FUN_0010323c();`
- `__src = (char *) (... GetByteArrayElements ...);`
- `strncpy((char *)&DAT_00115038, __src, 0x18);`
- `... ReleaseByteArrayElements(...);`

- `DAT_00115054 = DAT_00115054 + 1;`

This indicates that `init()` performs two important operations: Executes the native anti-debugging routine.

Copies the supplied XOR key into: `DAT_00115038`

The value of: `DAT_00115054`

This explains why `bar()` checks: `if (DAT_00115054 == 2)`

The initialization state is therefore part of the validation flow.

4.7 Native Anti-Debugging Analysis

The `init()` function calls: `FUN_0010323c()`;

The function initially appeared complex, but further analysis revealed a recognizable anti-debugging pattern.

The function uses:

- `fork()`
- `getppid()`
- `ptrace(PTRACE_ATTACH, ...)`
- `waitpid()`
- `ptrace(PTRACE_CONT, ...)`
- `pthread_create()`
- `_exit()`

The key operation is: `ptrace(PTRACE_ATTACH, parent_pid, ...)`

The native code creates a child process and attempts to attach to the parent process.

This is an anti-debugging technique intended to interfere with another debugger attempting to attach to the application.

4.8 Native Anti-Debugging Bypass

The native anti-debugging function was identified as: `FUN_0010323c`

The function was patched so that its execution immediately returns instead of performing the original anti-debugging logic.

The modification effectively changed the function's entry behavior from: `sub sp, sp, #...`
to: `ret`

This prevents the native anti-debugging routine from executing operations including:

- fork()
- ptrace()
- waitpid()
- pthread_create()

The remainder of the application can therefore continue without executing the original native anti-debugging logic.

4.9 Java Root and Debugger Detection

The original MainActivity.onCreate() contained the following root-detection logic:

```
if (b.a() || b.b() || b.c()) {
    a("Root detected!");
}
```

The application also checked whether the application was debuggable:

```
if (a.a(getApplicationContext())) {
    a("App is debuggable!");
}
```

An additional asynchronous debugger-detection routine was also identified:

```
while (!Debug.isDebuggerConnected()) {
    SystemClock.sleep(100L);
}
```

Once a debugger was detected, the application displayed: Debugger detected!

These mechanisms were removed from the patched Smali implementation. This allowed the application to start without triggering the original Java-level security alerts.

4.10 baz() and APK Integrity Verification

The third native method identified during the analysis was: private native long baz();

Unlike bar() and init(), the baz() method does not appear to process user input.

Its JNI implementation was identified as:

- undefined8
- Java_sg_vantagepoint_uncrackable3_MainActivity_baz(void) {return 0x18110e3;}

The Java code uses this value when validating the CRC of classes.dex:

- ZipEntry entry3 = zipFile.getEntry("classes.dex");
- Log.v(TAG, "CRC[classes.dex] = " + entry3.getCrc());
- if (entry3.getCrc() != baz()) {tampered = 31337;}

The purpose of this mechanism is to detect modification of the application's classes.dex. However, during the successful solution, this mechanism was not patched.

4.11 Secret Recovery

Analysis of the native password-verification routine ultimately revealed that the application expects a 24-byte secret.

The recovered secret was: making owasp great again

The recovered password was subsequently entered into the application and successfully validated.

The application displayed the expected: Success!

This confirmed that the static analysis of the native password-verification routine was correct.

4.12 Rebuilding and Signing

After the Java/Smali and native modifications were completed, the APK was rebuilt using Apktool.

The resulting APK was then signed with a locally generated Android keystore.

Signature verification was performed using: `apksigner verify --verbose UnCrackable-Level3-signed.apk`

The signed APK was subsequently installed into the Android virtual machine for dynamic validation.

4.13 Dynamic Validation

The final patched application was tested in the Android virtual machine.

4.13.1 Test 3 — Root Detection

Result: Bypassed

4.13.2 Test 4 — Debuggable Detection

Result: Bypassed

4.13.3 Test 5 — Native Anti-Debugging

Result: Bypassed

4.13.4 Test 6 — Password Verification

Result: PASS

4.13.5 Test 7 — Correct Secret

Input: making owasp great again

Result: PASS

5. Recommendation

For a production Android application:

- Do not rely on client-side root and debugger detection as a security boundary, as these mechanisms can be bypassed through Java/Smali and native code modification.
- Do not store sensitive secrets in native libraries, native code can still be reverse engineered and hardcoded secrets can be recovered.
- Do not rely solely on native code to protect security-sensitive logic, JNI and native implementations can be analyzed and modified.
- Move password verification and sensitive validation to a trusted backend, rather than performing critical verification entirely on the client.
- Use code obfuscation and native code hardening as defense-in-depth, but do not treat them as a replacement for secure application architecture.

6. Conclusion

The UnCrackable-Level3 application was successfully analyzed using static analysis, Smali analysis, native binary analysis, APK modification, and dynamic testing. The assessment demonstrated that:

- Root and Java debugger detection mechanisms could be identified and bypassed.
- Native anti-debugging implemented using ptrace() could be identified and bypassed.
- The XOR-based native password verification mechanism could be reverse engineered.
- The hardcoded XOR key and native verification logic could be analyzed to recover the secret making owasp great again.
- The modified APK could be rebuilt, signed, and successfully installed on the Android 9 testing environment.
- The recovered secret was successfully accepted by the application's native password verification mechanism.
- The baz() APK integrity mechanism was analyzed and left unchanged during the successful test.